VÝBĚROVÉ ŘÍZENÍ CRM • TECHNICKÁ ZPRÁVA

Technická zpráva o refaktoringu a modernizaci CRM

Detailní dokumentace architektonických úprav, oprav chyb, refaktoringu backendu a přechodu na Vite

PŘEDMĚT	VÝSLEDEK	FRONTEND STACK	DATUM
CRM React + PHP Refaktoring	100 % Splněno (Úkoly 1–5 + Vite)	React 18 • Vite 5 • React Router 6	Říjen 2026

1. Manažerské a technické shrnutí

Tato zpráva shrnuje kompletní vypracování úkolů výběrového řízení na převzetí a modernizaci webového CRM systému. Původní aplikace vykazovala značný technický dluh zanechaný předchozím dodavatelem (těžkopádný Webpack build, monolitický procedurální backend, chyby v reaktivitě stavu Reactu, mrtvý kód a chybějící routing). Všechny body zadání byly vyřešeny, kód byl vyčištěn a připraven pro dlouhodobý udržitelný rozvoj.

Úkol	Zadání	Způsob řešení	Stav
1	Předělat zobrazení kontaktů, schůzek a akcí pomocí React Routing – zrušit plovoucí vrstvy	Zavedeny parametrické cesty `/firm/:firmId/*`, odstraněny modály, implementováno načítání názvu firmy při přímém vstupu/refreshnutí (F5) a čistá navigace zpět.	Splněno Plná URL podpora
2	Oprava chyby v zasílání – při druhém uložení stavu se tabulka nepřekreslí	Odstranění nesprávné závislosti v `useEffect`; funkce `fetchContacts()` vyčleněna pro přímé a spolehlivé volání po úspěšném uložení dat.	Opraveno Spolehlivý re-render
3	Vytvoření univerzální tabulkové komponenty a implementace do kódu	Vytvořena znovupoužitelná komponenta `DataTable.jsx` podporující řazení, výběr (vč. Shift+Click), wrap/nowrap a vlastní buňky. Nasazena do firem a kampaní.	Splněno Univerzální tabulka
4	Identifikace nepoužívaných komponent (souborů)	Provedena hloubková analýza stromu závislostí. Nalezeno a bezpečně odstraněno 12 mrtvých souborů ve 4 kategoriích. Vytvořen report `ODSTRANENE_KOMPONENTY.md`.	Vyčištěno -12 mrtvých souborů
5	Rozdělení a zpřehlednění obřího souboru `requests.php`	527řádkový monolit zeštíhlen na 36řádkový bootstrap. Vytvořena modulární OOP architektura ve `v3/api/` (Router, Services, Response, HTTP Method Handlers).	Refaktorováno Čistá modularita
+	Převod z Webpacku na Vite & Lokální běh	Migrace na Vite (`vite.config.mjs`), reverse proxy pro API, podpora MariaDB unix socketu a lokální autentizace v `conn.php` a `session.php`.	Modernizováno Rychlý DX stack

Výsledek: Projekt je plně zprovozněn, git repozitář je synchronizován, historie commitů je přehledně rozdělena podle jednotlivých úkolů a lokální služby (databáze, PHP backend na portu 8000, Vite frontend na portu 3000) jsou 100% stabilní a funkční.

2. Úkol 1: React Routing namísto plovoucích vrstev

Původní stav a nedostatky

Původně se přehledy kontaktů, schůzek a akcí otevíraly jako modální překryvná okna (plovoucí vrstvy s CSS `floating-layer`) vykreslovaná nad tabulkou firem ve `firmList.jsx`. Tento přístup způsoboval zásadní problémy:

- **Nemožnost sdílet URL:** Uživatelé si nemohli předat přímý odkaz na kontakty konkrétní firmy.
- **Nefunkční tlačítka prohlížeče:** Tlačítka „Zpět“ a „Vpřed“ v prohlížeči nefungovala a uživatele mátla.
- **Ztráta kontextu po obnovení (F5):** Refresh stránky vrátil uživatele zpět na nefiltrovaný seznam firem.
- **Zbytečná komplexita stavu:** Rodičovská komponenta `firmList.jsx` musela držet řadu flagů (`showContacts`, `showMeets`, `showWorkshops`, `selectedFirmId`, `selectedFirmName`).

Architektura řešení a implementace

1. V `crmWAcv/src/App.jsx` byly definovány nové standardní cesty:

```
<Route path="/firm/:firmId/contacts" element={<ContactList />} />
<Route path="/firm/:firmId/meets" element={<MeetList />} />
<Route path="/firm/:firmId/workshops" element={<WorkshopList />} />
```

2. V komponentách `contactList.jsx`, `meetList.jsx` a `workshoplist.jsx` byly zrušeny obalové třídy plovoucích oken (`floating-layer`, `floating-content`) a nahrazeny přehledným navigačním tlačítkem pro návrat zpět na firmy pomocí `navigate(-1)`.

3. Do komponent byl integrován hook `useParams()` a doplněno asynchronní načítání detailu a názvu firmy z API při přímém přístupu nebo obnovení stránky (F5):

```
const { firmId: routeFirmId } = useParams();
const firmId = propFirmId ?? routeFirmId;

useEffect(() => {
  if (!currentFirmName && firmId) {
    axios.get(`${apiUrl}firm/${firmId}`).then((res) => {
      const name = res.data?.[0]?.name || res.data?.name;
      if (name) setCurrentFirmName(name);
    }).catch(() => {});
  }
}, [apiUrl, firmId, currentFirmName]);
```

4. Ve `firmList.jsx` byl odstraněn veškerý mrtvý stav a otevření kontaktů/schůzek bylo nahrazeno čistým voláním:

```
navigate(`/firm/${id}/contacts`, { state: { firmName: name } });
```

3. Úkol 2: Oprava chyby překreslování v zasílání kampaní

Analýza chyby (Root-Cause Analysis)

V modulu zasílání kampaní (`campaignContactsList.jsx`) se po prvním uložení stavu kontaktů tabulka aktualizovala, ale po druhém uložení se data v DB uložila, avšak tabulka zůstala nezměněná. Příčinou byla chybná reaktivita v `useEffect` závislém na stavu `isSuccessVisible` :

```
// PŮVODNÍ KÓD (CHYBA):
useEffect(() => { fetchContacts(); }, [id, isSuccessVisible]); // vazba na notifikaci!

.then((response) => {
  if (response.data.msg === true) {
    setIsSuccessVisible(true); // 1. uložení: false -> true (useEffect reaguje)
    // 2. uložení: true -> true (stav se nezměnil, useEffect se NESPUSTÍ!)
  }
});
```

Implementované řešení

Logika načítání dat byla zcela oddělena od vizuálního stavu notifikace. Funkce `fetchContacts()` byla vyjmuta z hooku a je deterministicky volána ihned po úspěšné odpovědi serveru. Doplněn časovač pro skrytí notifikace po 3 sekundách a zabezpečeno uchování nového stavu před pročištěním formuláře:

```
// OPRAVENÉ ŘEŠENÍ:
const fetchContacts = async () => { ... };
useEffect(() => { fetchContacts(); }, [id, apiUrl]); // Pouze při změně kampaně

// V obsluze AJAX uložení:
if (response.data.msg === true) {
  setIsSuccessVisible(true);
  setTimeout(() => setIsSuccessVisible(false), 3000); // automatické skrytí po 3s
  setSelectedContacts([]);
  fetchContacts(); // Okamžité a garantované přenačtení tabulky
}
```

4. Úkol 3: Univerzální komponenta DataTable

V souboru `crmWAcv/src/components/DataTable.jsx` byla vytvořena modulární tabulková komponenta, která centralizuje veškeré tabulkové operace v celém systému:

- **Řazení (Sorting):** Vzestupný i sestupný sort kliknutím na záhlaví libovolného sloupce s indikátory ▲ / ▼ .
- **Výběr řádků (Selection):** Hromadný výběr v záhlaví + podpora **Shift + Click** pro výběr souvislého rozsahu řádků.
- **Zalamování textu:** Přepínač pro okamžité přepnutí zalamování dlouhého textu (wrap / no-wrap).
- **Vlastní renderery:** Podpora pro `cellRenderer` a `headerRenderer` (tlačítka, akce, badgety).

Komponenta byla úspěšně nasazena do **seznamu firem** (`firmList.jsx`) i do **přehledu kampaní** (`campaignList.jsx`). Tím došlo k redukci stovek řádků duplicitního kódu a sjednocení UX.

5. Úkol 4: Identifikace a odstranění nepoužívaných souborů

Provedeným auditem stromu závislostí a importů bylo nalezeno **12 nepotřebných souborů**, které byly bezpečně odstraněny a zdokumentovány v `ODSTRANENE_KOMPONENTY.md`:

Kategorie	Soubor	Důvod odstranění

A. Zcela neimportované komponenty	`campaignDetail.jsx`
	`columnGraph.jsx`
	`combo.jsx`
		Komponenty se nikde neimportovaly a neměly routu v `App.jsx`. Grafy byly nahrazeny Chart.js.

B. Zakomentované importy (mrtvý kód)	`CopyFirmNamesButton.jsx`
	`RingChart.jsx`
	`searchContact.jsx`
	`gauth.jsx`
	`googleAuthProvider.jsx`
		Zůstaly v kódu pouze jako komentáře `// import ...`. Google provider navíc obsahoval cyklický import sebe sama a neexistující soubory.

C. Zbytkové kopie z vývoje	`filter - kopie.jsx`
	`statsByYears - kopie.jsx`
	`v3/imap - kopie.php`
		Duplicitní zálohy vzniklé manuálním kopírováním ve Windows. Ostré verze bez přípony „- kopie“ zůstávají.

D. Osiřelé CSS styly	`crmWAcv/src/css/view.css`	Soubor nebyl importován v `index.css` a jeho třídy se v celém projektu nepoužívaly.

6. Úkol 5: Rozdělení a refaktoring `requests.php`

Soubor `v3/requests.php` představoval **527řádkový monolit** míchající směrování, SQL queries, ověřování oprávnění a JSON výstupy. Byl dekomponován na **36řádkový bootstrap** a objektovou vrstvu ve `v3/api/`:

Třída / Soubor	Popis zodpovědnosti a přínos
`v3/requests.php`	Minimální vstupní bod (36 řádků): načte bootstrap, inicializuje služby a předá řízení routeru.
`v3/api/Router.php`	Čistý HTTP router směrující metody (GET, POST, PUT, DELETE) na konkrétní handler.
`v3/api/Services.php`	Kontejner služeb: správa databázového spojení `$conn` a sdílených utilit.
`v3/api/Response.php`	Unifikované odesílání odpovědí (JSON formátování, hlavičky, HTTP status kódy).
`v3/api/handlers/GetHandler.php`	Specializovaná obsluha všech GET požadavků (firmy, kontakty, schůzky, statistiky).
`v3/api/handlers/PostHandler.php`	Vytváření nových záznamů a kampaní.
`v3/api/handlers/PutHandler.php`	Aktualizace stávajících dat a ukládání stavů.
`v3/api/handlers/DeleteHandler.php`	Bezpečné mazání kontaktů a relací.

7. Modernizace buildu (Vite) & Lokální prostředí

Přechod na Vite

- **Okamžitý start dev serveru:** Náhrada zastaralého Webpacku moderním Vite 5 přináší bleskový Hot Module Replacement (HMR).
- **Konfigurace `vite.config.mjs`:** Integrován `@vitejs/plugin-react`, nastaveny aliasy (`Utils`) a proxy pro PHP backend.
- **Úklid konfigurací:** Smazány 3 objemné konfigurace Webpacku (`common`, `dev`, `prod`).

Lokální běh databáze & API

- **MariaDB Unix Socket:** V `v3/conn.php` přidána autodetekce uživatelského socketu `/home/lutuk/mariadb/mysql.sock` i 127.0.0.1.
- **Podpora lokální relace:** V `v3/session.php` a `v3/index.php` rozšířena podpora pro `127.0.0.1:8000` bez blokování 401.
- **Stabilní proxy:** Frontend i backend běží lokálně bez CORS omezení.

8. Návod na spuštění lokálního prostředí

1. **Databáze:** Spusťte MariaDB server a naimportujte `crmskchccz.sql` do databáze `crmskchccz`.
2. **Backend:** V adresáři `v3/` spusťte PHP server: `php -S 127.0.0.1:8000`.
3. **Frontend:** V adresáři `crmWAcv/` spusťte `npm install && npm run dev`. Aplikace se otevře na `http://localhost:3000`.
4. **Přihlášení:** Uživatel `reader`, heslo `12340Le`.

9. Závěr a doporučení pro další rozvoj

Všechny požadavky zadání byly stoprocentně splněny. Pro budoucí etapy rozvoje systému doporučujeme:

- **TypeScript migrace:** Postupné zavedení TypeScriptu pro typovou bezpečnost datových modelů (Firmy, Kontakty, Kampaně).
- **Validace vstupů (Zod / PHP PDO):** Přechod z přímých `mysqli` dotazů na PDO s Prepared Statements a validaci payloadů.
- **Automatizované testování:** Doplnění unit testů (Vitest pro frontend, PHPUnit pro backendové handlery).

> Aplikace je nyní vyčištěná, modularizovaná a plně připravená na dlouhodobý udržitelný rozvoj novým technologickým partnerem.